

Home Assignment — Satellite Camera Data Manager

ISI · Embedded Software · time-box ≈ 3–4 hours · runs on a normal PC (no hardware)

This exercise is small and open-ended on purpose. We care much less about a "finished product" and much more about **how you structure software, how you handle the awkward cases, and how you explain your choices**. A modest, well-thought-out, well-argued solution beats a big shaky one.

The situation (plain and simple)

A small satellite carries a camera. As it flies around the Earth it takes pictures on a schedule — each picture is a file with a **size** and an **importance** (high / medium / low).

- The satellite's **onboard storage is limited** — it can't keep every picture.
- It can only send pictures to Earth while it's **passing over the ground station** — a couple of short windows each orbit — and the link has a **limited speed**, so it can't send everything either.

Your software runs on the satellite and decides, as time goes by: **which pictures to keep, which to discard when storage is full, and which to send first during each window**.

What to build

A program that plays through one orbit and, over time:

- Takes each picture at its scheduled time and tries to store it.
- When storage is full, decides what to discard (or whether to skip the new one) — **your policy**.
- During each ground-station window, sends stored pictures to Earth up to the link speed, in an order **you** choose.
- At the end, prints a clear timeline of what happened plus a summary: how many pictures were taken, sent, and discarded; total MB sent; the highest storage level reached.

Two input files are provided:

pictures.csv	meaning
take_at_min	minute the picture is taken (0 = start of the run)
size_mb	file size, in MB
importance	high / medium / low

passes.csv	meaning
window_start_min , window_end_min	the minutes during which the ground station is reachable
link_speed_mb_per_min	how many MB can be sent per minute during that window

Assume onboard storage holds **512 MB** (make it easy to change). Heads-up: the schedule deliberately creates **more pictures than can be stored or sent** — so the interesting part is your keep / discard / send-order decisions.

Fixed rules (please stay inside these)

- Use **whatever language and tools you're most comfortable with**. It just needs to run on a PC and read the two CSV files.
- Deliver: your source + a short `README` (how to run it) + whatever you'll present.
- Same inputs should give the same result, so you can demo and repeat a run.

Please do NOT build these

- No orbits/physics — the schedule and windows are given to you.
- No graphical interface, no networking, no real hardware.
- No saving state between runs, no login/security.
- Don't over-build. If a piece would take more than ~45 minutes, **leave a simple stub and note it as a tradeoff**.

Left up to you (on purpose)

There's no single right answer here — and we'll ask you to explain these:

- What to discard when storage is full (by importance? by age? by size? something smarter?).
- What to send first when a window is too short to send everything.
- How you organize the code, handle bad/edge cases, and check that it works.

Using AI tools

Use Claude / Copilot / ChatGPT / anything — we do too. If you do, just **note where you used it and what you changed**. Fair warning: the presentation and questions assume you can explain and modify every part of what you hand in.

The presentation — this is the main thing (~15–20 min)

- Walk us through your design and **why** — especially how you decided what to discard and what to send first.

- Run it live, and show what happens when storage fills up and a window runs out of time.
- What was the trickiest part, and how did you handle it?
- What would you add with two more days, and what did you deliberately leave out?

Aim for ~3–4 hours. If you run over, stop and bring notes on what you'd do next — that conversation counts too. Good luck, and enjoy it.